

WEATHER-BASED ACTIVITY RECOMMENDATION SYSTEM

Project Report

Submitted by: BhanuBindu D

REG NO:113025148013

Department: B.E / COMPUTER SCIENCE ENGINEERING(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

1. Abstract

The Weather-Based Activity Recommendation System is a Python-based, rule-driven Artificial Intelligence application that recommends suitable indoor or outdoor activities according to weather conditions. The system accepts temperature, weather condition, humidity, and wind speed as inputs. It validates the inputs and processes them through a rational decision-making agent. The agent first checks severe weather conditions, then checks rainfall and thermal comfort, and finally produces activities with a logical explanation. The project also includes a Flask web application for connecting the recommendation engine with a user interface.

2. Introduction

Weather affects daily planning, exercise, travel, recreation, and productivity. People may find it difficult to decide whether an outdoor activity is suitable under particular weather conditions. This project solves that problem by using a knowledge-based rational agent. Instead of giving a random suggestion, the agent uses predefined rules to prioritize safety, comfort, and suitability.

3. Objectives

- To create a weather-based activity recommendation system using Python.
- To accept temperature, condition, humidity, and wind-speed inputs.
- To validate weather inputs before processing them.
- To recommend safe and suitable indoor or outdoor activities.
- To provide an understandable explanation for every recommendation.
- To connect the Python recommendation engine with a Flask web interface.
- To test the agent using different weather scenarios.

4. Technologies Used

- Python – Core programming and decision-making logic.
- Flask – Web application framework.
- HTML – User interface structure.
- CSS – Interface styling.

- JavaScript – Front-end interaction and API communication.
- Google Colab – Running the Python code and collecting outputs.
- GitHub – Project source-code storage and pipeline sharing.

5. Artificial Intelligence Concept: PEAS

PEAS Element	Project Description
Performance Measure	Safety, physical comfort, and appropriateness of activities.
Environment	Current weather and atmospheric conditions.
Actuators	Recommended activities, indoor/outdoor classification, and explanation.
Sensors	Temperature, weather condition, humidity, and wind speed.

6. System Workflow

- The user enters weather details.
- WeatherPercept stores and validates the input values.
- ActivityAgent receives the validated weather percept.
- The safety filter checks storms, high wind, extreme heat, and freezing temperatures.
- The system checks rainfall and thermal comfort.
- The agent selects suitable activities from its knowledge base.
- The final result contains classification, summary, explanation, and recommended activities.

7. Main Python Modules

7.1 WeatherPercept

The WeatherPercept class represents the sensory input received by the agent. Its constructor converts temperature, humidity, and wind speed into numeric values and standardizes the weather condition. The validate() method checks realistic ranges and returns a list of error messages when invalid data is entered.

7.2 ActivityAgent

The ActivityAgent class contains the activity knowledge base and the evaluate() method. The knowledge base stores outdoor and indoor activities with names, icons, and descriptions. The evaluate() method applies the decision rules in sequence and returns a dictionary containing the recommendation and reasoning.

7.3 Flask Application

The app.py file connects the agent to a web interface. It receives weather values from the front end, creates a WeatherPercept object, validates the values, calls the agent, and returns the result as JSON. This allows the recommendation engine to be used through a browser-based application.

8. Decision Rules

- Stormy weather is treated as severe because of lightning, heavy rain, and low visibility.
- Wind speed of 45 km/h or above is treated as hazardous.
- Temperature of 40°C or above is treated as extreme heat.

- Temperature of 0°C or below is treated as freezing weather.
- Rainy weather recommends indoor activities because wet ground can be slippery.
- Temperature of 33°C or above, or temperature of at least 30°C with humidity of 75% or above, favors indoor activities.
- Otherwise, the weather is classified as outdoor favored.

9. Sample Input and Output

Sample input used in Google Colab:

Temperature: 25.0 °C
Condition: Sunny
Humidity: 50.0 %
Wind Speed: 10.0 km/h

Observed output:

Classification: Outdoor Favored (Pleasant & Comfortable)
Summary: Outdoor Recommended
Explanation: Clear sunny skies and moderate temperature
Recommended Activities:
- Cycling / Biking
- Jogging or Running
- Picnic in the Park
- Outdoor Sports (Football/Cricket/Tennis)

10. Advantages

- Simple and easy-to-understand decision-making logic.
- Prioritizes safety before recommending activities.
- Provides explanations instead of only giving an activity name.
- Works with both command-line Python execution and a Flask web interface.
- Can be tested easily with different weather inputs.

11. Limitations

- The current version uses user-entered weather data.
- It does not automatically fetch live weather information.
- The recommendations are based on predefined rules rather than a trained machine-learning model.
- The system does not consider personal preferences, medical conditions, or location-specific risks.

12. Future Enhancements

- Connect the system to a live weather API.
- Add location-based weather detection.
- Include user preferences such as age, interests, and fitness level.
- Use machine learning to learn from user feedback.
- Add notifications and weather alerts.

- Deploy the Flask application online.

13. Conclusion

The Weather-Based Activity Recommendation System successfully demonstrates a rational AI agent using Python. It accepts weather percepts, validates the input, applies safety and comfort rules, and recommends suitable activities with explanations. The project also demonstrates how a Python decision engine can be connected to a Flask web interface. It is useful as an educational project for understanding intelligent agents, rule-based reasoning, validation, and basic web integration.

14. Project Files

- agent.py – Weather percept class, activity knowledge base, and recommendation logic.
- app.py – Flask web application and API routes.
- test_agent.py – Tests for recommendation rules.
- test_app.py – Tests for Flask routes and input handling.
- templates/index.html – Web page structure.
- static/js/app.js – Front-end interaction.
- requirements.txt – Python dependency list.
- README.md – Project documentation.